

Pathfinding Algorithms

Core DSA Techniques with Java & C Implementations

August 8, 2026

What this document covers: the main graph pathfinding algorithms taught in Data Structures & Algorithms courses — Breadth-First Search, Depth-First Search, Dijkstra's Algorithm, Bellman–Ford, Floyd–Warshall, and A* Search — each with a short explanation, complexity summary, and complete, compilable **Java** and **C** implementations.

Contents

1	Overview of Pathfinding Algorithms	2
2	Breadth-First Search (BFS)	3
2.1	Java Implementation	3
2.2	C Implementation	4
3	Depth-First Search (DFS)	6
3.1	Java Implementation	6
3.2	C Implementation	7
4	Dijkstra's Algorithm	9
4.1	Java Implementation	9
4.2	C Implementation	10
5	Bellman–Ford Algorithm	12
5.1	Java Implementation	12
5.2	C Implementation	13
6	Floyd–Warshall Algorithm	15
6.1	Java Implementation	15
6.2	C Implementation	16
7	A* Search Algorithm	17
7.1	Java Implementation	17
7.2	C Implementation	18
8	Summary & Algorithm Selection Guide	21

1 Overview of Pathfinding Algorithms

Pathfinding algorithms operate on a **graph** $G = (V, E)$, where V is a set of vertices (nodes) and E is a set of edges (connections), to find a route between a source and a destination. The choice of algorithm depends on whether edges are weighted, whether weights can be negative, and whether we need one shortest path or all pairs of shortest paths.

Algorithm	Weighted?	Negative Weights?	Finds
BFS	No (unweighted)	N/A	Shortest path (fewest edges)
DFS	No	N/A	A path (not necessarily shortest)
Dijkstra's	Yes	No	Single-source shortest paths
Bellman–Ford	Yes	Yes	Single-source shortest paths
Floyd–Warshall	Yes	Yes (no neg. cycles)	All-pairs shortest paths
A* Search	Yes	No (needs heuristic)	Single-pair shortest path (optimal)

2 Breadth-First Search (BFS)

BFS explores a graph level by level using a queue. On an **unweighted** graph, the first time BFS reaches a node, it does so via the shortest possible path (fewest edges), which makes BFS the natural shortest-path algorithm for unweighted graphs and grids.

Time Complexity	$O(V + E)$
Space Complexity	$O(V)$
Guarantees Shortest Path	Yes, for unweighted graphs
Typical Use Case	Shortest path in unweighted graphs / grids, level-order traversal

2.1 Java Implementation

```

1  import java.util.*;
2
3  public class BFS {
4
5      // Adjacency list representation of an unweighted graph
6      private final Map<Integer, List<Integer>> graph = new HashMap<>();
7
8      public void addEdge(int u, int v) {
9          graph.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
10         graph.computeIfAbsent(v, k -> new ArrayList<>()).add(u); // undirected
11     }
12
13     // Returns the shortest path (fewest edges) from src to dest, or empty list
14     public List<Integer> shortestPath(int src, int dest) {
15         Map<Integer, Integer> parent = new HashMap<>();
16         Set<Integer> visited = new HashSet<>();
17         Queue<Integer> queue = new LinkedList<>();
18
19         queue.add(src);
20         visited.add(src);
21         parent.put(src, -1);
22
23         while (!queue.isEmpty()) {
24             int node = queue.poll();
25             if (node == dest) break;
26
27             for (int neighbor : graph.getOrDefault(node, Collections.emptyList())) {
28                 if (!visited.contains(neighbor)) {
29                     visited.add(neighbor);
30                     parent.put(neighbor, node);
31                     queue.add(neighbor);
32                 }
33             }
34         }
35
36         if (!visited.contains(dest)) return Collections.emptyList();
37
38         // Reconstruct path from dest back to src
39         LinkedList<Integer> path = new LinkedList<>();
40         for (int at = dest; at != -1; at = parent.get(at)) {
41             path.addFirst(at);
42         }

```

```
43     return path;
44 }
45
46 public static void main(String[] args) {
47     BFS bfs = new BFS();
48     bfs.addEdge(0, 1);
49     bfs.addEdge(0, 2);
50     bfs.addEdge(1, 3);
51     bfs.addEdge(2, 3);
52     bfs.addEdge(3, 4);
53
54     List<Integer> path = bfs.shortestPath(0, 4);
55     System.out.println("Shortest path (BFS): " + path);
56 }
57 }
```

Listing 1: BFS Shortest Path – Java

2.2 C Implementation

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAX_V 100
5
6  int adj[MAX_V][MAX_V];    // adjacency matrix
7  int numVertices;
8
9  int parent[MAX_V];
10 int visited[MAX_V];
11 int queue[MAX_V];
12 int front, rear;
13
14 void enqueue(int v) { queue[rear++] = v; }
15 int dequeue(void) { return queue[front++]; }
16 int isEmpty(void) { return front == rear; }
17
18 void addEdge(int u, int v) {
19     adj[u][v] = 1;
20     adj[v][u] = 1; // undirected
21 }
22
23 // Prints the shortest path from src to dest using BFS
24 void bfsShortestPath(int src, int dest) {
25     for (int i = 0; i < numVertices; i++) {
26         visited[i] = 0;
27         parent[i] = -1;
28     }
29     front = rear = 0;
30
31     visited[src] = 1;
32     enqueue(src);
33
34     while (!isEmpty()) {
35         int node = dequeue();
36         if (node == dest) break;
37
38         for (int neighbor = 0; neighbor < numVertices; neighbor++) {
39             if (adj[node][neighbor] && !visited[neighbor]) {
40                 visited[neighbor] = 1;
41                 parent[neighbor] = node;
42                 enqueue(neighbor);
43             }
44         }
45     }
46 }
```

```
43     }
44 }
45 }
46
47 if (!visited[dest]) {
48     printf("No path exists from %d to %d\n", src, dest);
49     return;
50 }
51
52 int path[MAX_V], count = 0;
53 for (int at = dest; at != -1; at = parent[at]) {
54     path[count++] = at;
55 }
56
57 printf("Shortest path (BFS): ");
58 for (int i = count - 1; i >= 0; i--) {
59     printf("%d", path[i]);
60     if (i > 0) printf(" -> ");
61 }
62 printf("\n");
63 }
64
65 int main(void) {
66     numVertices = 5;
67     addEdge(0, 1);
68     addEdge(0, 2);
69     addEdge(1, 3);
70     addEdge(2, 3);
71     addEdge(3, 4);
72
73     bfsShortestPath(0, 4);
74     return 0;
75 }
```

Listing 2: BFS Shortest Path – C

3 Depth-First Search (DFS)

DFS explores as far as possible along a branch before backtracking, typically implemented with recursion or an explicit stack. It is used to find *a* path (or detect connectivity/cycles), but the path found is generally **not** the shortest one.

Time Complexity	$O(V + E)$
Space Complexity	$O(V)$ (recursion stack)
Guarantees Shortest Path	No
Typical Use Case	Connectivity, cycle detection, topological sort, maze solving

3.1 Java Implementation

```

1  import java.util.*;
2
3  public class DFS {
4
5      private final Map<Integer, List<Integer>> graph = new HashMap<>();
6
7      public void addEdge(int u, int v) {
8          graph.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
9          graph.computeIfAbsent(v, k -> new ArrayList<>()).add(u); // undirected
10     }
11
12     // Finds *a* path from src to dest (not necessarily the shortest)
13     public List<Integer> findPath(int src, int dest) {
14         List<Integer> path = new ArrayList<>();
15         Set<Integer> visited = new HashSet<>();
16         if (dfs(src, dest, visited, path)) {
17             return path;
18         }
19         return Collections.emptyList();
20     }
21
22     private boolean dfs(int node, int dest, Set<Integer> visited, List<Integer>
23         path) {
24         visited.add(node);
25         path.add(node);
26
27         if (node == dest) return true;
28
29         for (int neighbor : graph.getOrDefault(node, Collections.emptyList())) {
30             if (!visited.contains(neighbor)) {
31                 if (dfs(neighbor, dest, visited, path)) return true;
32             }
33         }
34
35         path.remove(path.size() - 1); // backtrack
36         return false;
37     }
38
39     public static void main(String[] args) {
40         DFS dfs = new DFS();
41         dfs.addEdge(0, 1);
42         dfs.addEdge(0, 2);
43         dfs.addEdge(1, 3);
44     }
45 }

```

```
43         dfs.addEdge(2, 3);
44         dfs.addEdge(3, 4);
45
46         List<Integer> path = dfs.findPath(0, 4);
47         System.out.println("A path found by DFS: " + path);
48     }
49 }
```

Listing 3: DFS Path Finding – Java

3.2 C Implementation

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAX_V 100
5
6  int adj[MAX_V][MAX_V];
7  int numVertices;
8  int visited[MAX_V];
9  int path[MAX_V];
10 int pathLen;
11
12 void addEdge(int u, int v) {
13     adj[u][v] = 1;
14     adj[v][u] = 1; // undirected
15 }
16
17 // Returns 1 if a path from node to dest is found (stored in path[])
18 int dfs(int node, int dest) {
19     visited[node] = 1;
20     path[pathLen++] = node;
21
22     if (node == dest) return 1;
23
24     for (int neighbor = 0; neighbor < numVertices; neighbor++) {
25         if (adj[node][neighbor] && !visited[neighbor]) {
26             if (dfs(neighbor, dest)) return 1;
27         }
28     }
29
30     pathLen--; // backtrack
31     return 0;
32 }
33
34 int main(void) {
35     numVertices = 5;
36     addEdge(0, 1);
37     addEdge(0, 2);
38     addEdge(1, 3);
39     addEdge(2, 3);
40     addEdge(3, 4);
41
42     pathLen = 0;
43     for (int i = 0; i < numVertices; i++) visited[i] = 0;
44
45     if (dfs(0, 4)) {
46         printf("A path found by DFS: ");
47         for (int i = 0; i < pathLen; i++) {
48             printf("%d", path[i]);
49             if (i < pathLen - 1) printf(" -> ");
50         }
51     }
```

```
51     printf("\n");
52 } else {
53     printf("No path exists.\n");
54 }
55
56 return 0;
57 }
```

Listing 4: DFS Path Finding – C

4 Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest path from a single source to all other vertices in a graph with **non-negative** edge weights. It repeatedly selects the unvisited vertex with the smallest known distance (using a min-priority queue) and relaxes its outgoing edges.

Time Complexity	$O((V + E) \log V)$ with a binary heap
Space Complexity	$O(V + E)$
Guarantees Shortest Path	Yes (non-negative weights only)
Typical Use Case	Road networks, network routing (e.g. OSPF), weighted shortest path

4.1 Java Implementation

```

1  import java.util.*;
2
3  public class Dijkstra {
4
5      static class Edge {
6          int to, weight;
7          Edge(int to, int weight) { this.to = to; this.weight = weight; }
8      }
9
10     private final Map<Integer, List<Edge>> graph = new HashMap<>();
11
12     public void addEdge(int u, int v, int weight) {
13         graph.computeIfAbsent(u, k -> new ArrayList<>()).add(new Edge(v, weight));
14         graph.computeIfAbsent(v, k -> new ArrayList<>()).add(new Edge(u, weight));
15         // undirected
16     }
17
18     public int[] shortestPaths(int src, int numVertices) {
19         int[] dist = new int[numVertices];
20         Arrays.fill(dist, Integer.MAX_VALUE);
21         dist[src] = 0;
22
23         // Min-heap ordered by current known distance
24         PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a
25             -> a[1]));
26         pq.add(new int[]{src, 0});
27
28         boolean[] visited = new boolean[numVertices];
29
30         while (!pq.isEmpty()) {
31             int[] curr = pq.poll();
32             int u = curr[0];
33             if (visited[u]) continue;
34             visited[u] = true;
35
36             for (Edge e : graph.getOrDefault(u, Collections.emptyList())) {
37                 if (!visited[e.to] && dist[u] != Integer.MAX_VALUE
38                     && dist[u] + e.weight < dist[e.to]) {
39                     dist[e.to] = dist[u] + e.weight;
40                     pq.add(new int[]{e.to, dist[e.to]});
41                 }
42             }
43         }
44     }
45 }

```

```

41     }
42     return dist;
43 }
44
45 public static void main(String[] args) {
46     Dijkstra d = new Dijkstra();
47     d.addEdge(0, 1, 4);
48     d.addEdge(0, 2, 1);
49     d.addEdge(2, 1, 2);
50     d.addEdge(1, 3, 1);
51     d.addEdge(2, 3, 5);
52
53     int[] dist = d.shortestPaths(0, 4);
54     for (int i = 0; i < dist.length; i++) {
55         System.out.println("Distance from 0 to " + i + " = " + dist[i]);
56     }
57 }
58 }

```

Listing 5: Dijkstra's Algorithm – Java

4.2 C Implementation

```

1  #include <stdio.h>
2  #include <limits.h>
3
4  #define MAX_V 100
5  #define INF INT_MAX
6
7  int graph[MAX_V][MAX_V]; // adjacency matrix, 0 = no edge
8  int numVertices;
9
10 // Finds the unvisited vertex with the minimum distance
11 int minDistance(int dist[], int visited[]) {
12     int min = INF, minIndex = -1;
13     for (int v = 0; v < numVertices; v++) {
14         if (!visited[v] && dist[v] <= min) {
15             min = dist[v];
16             minIndex = v;
17         }
18     }
19     return minIndex;
20 }
21
22 void dijkstra(int src) {
23     int dist[MAX_V];
24     int visited[MAX_V];
25
26     for (int i = 0; i < numVertices; i++) {
27         dist[i] = INF;
28         visited[i] = 0;
29     }
30     dist[src] = 0;
31
32     for (int count = 0; count < numVertices - 1; count++) {
33         int u = minDistance(dist, visited);
34         if (u == -1) break; // remaining vertices unreachable
35         visited[u] = 1;
36
37         for (int v = 0; v < numVertices; v++) {
38             if (!visited[v] && graph[u][v] && dist[u] != INF
39                 && dist[u] + graph[u][v] < dist[v]) {

```

```
40         dist[v] = dist[u] + graph[u][v];
41     }
42 }
43 }
44
45 printf("Vertex \t Distance from Source %d\n", src);
46 for (int i = 0; i < numVertices; i++) {
47     if (dist[i] == INF)
48         printf("%d \t unreachable\n", i);
49     else
50         printf("%d \t %d\n", i, dist[i]);
51 }
52 }
53
54 int main(void) {
55     numVertices = 4;
56
57     // Initialize graph with 0 (no edge)
58     for (int i = 0; i < numVertices; i++)
59         for (int j = 0; j < numVertices; j++)
60             graph[i][j] = 0;
61
62     graph[0][1] = 4; graph[1][0] = 4;
63     graph[0][2] = 1; graph[2][0] = 1;
64     graph[2][1] = 2; graph[1][2] = 2;
65     graph[1][3] = 1; graph[3][1] = 1;
66     graph[2][3] = 5; graph[3][2] = 5;
67
68     dijkstra(0);
69     return 0;
70 }
```

Listing 6: Dijkstra's Algorithm – C

5 Bellman–Ford Algorithm

Bellman–Ford also computes single-source shortest paths, but unlike Dijkstra’s it tolerates **negative** edge weights and can detect **negative-weight cycles**. It works by relaxing every edge $V - 1$ times.

Time Complexity	$O(V \cdot E)$
Space Complexity	$O(V)$
Guarantees Shortest Path	Yes (detects negative cycles if present)
Typical Use Case	Graphs with negative weights, currency arbitrage detection, routing protocols (e.g. RIP)

5.1 Java Implementation

```

1  import java.util.*;
2
3  public class BellmanFord {
4
5      static class Edge {
6          int from, to, weight;
7          Edge(int from, int to, int weight) {
8              this.from = from; this.to = to; this.weight = weight;
9          }
10     }
11
12     private final List<Edge> edges = new ArrayList<>();
13     private final int numVertices;
14
15     public BellmanFord(int numVertices) {
16         this.numVertices = numVertices;
17     }
18
19     public void addEdge(int from, int to, int weight) {
20         edges.add(new Edge(from, to, weight)); // directed edge
21     }
22
23     public int[] shortestPaths(int src) {
24         int[] dist = new int[numVertices];
25         Arrays.fill(dist, Integer.MAX_VALUE);
26         dist[src] = 0;
27
28         // Relax all edges V - 1 times
29         for (int i = 1; i < numVertices; i++) {
30             for (Edge e : edges) {
31                 if (dist[e.from] != Integer.MAX_VALUE
32                     && dist[e.from] + e.weight < dist[e.to]) {
33                     dist[e.to] = dist[e.from] + e.weight;
34                 }
35             }
36         }
37
38         // Check for negative-weight cycles
39         for (Edge e : edges) {
40             if (dist[e.from] != Integer.MAX_VALUE
41                 && dist[e.from] + e.weight < dist[e.to]) {
42                 throw new IllegalStateException("Graph contains a negative-
                     weight cycle");

```

```

43     }
44 }
45
46     return dist;
47 }
48
49 public static void main(String[] args) {
50     BellmanFord bf = new BellmanFord(5);
51     bf.addEdge(0, 1, 6);
52     bf.addEdge(0, 2, 7);
53     bf.addEdge(1, 2, 8);
54     bf.addEdge(1, 3, 5);
55     bf.addEdge(1, 4, -4);
56     bf.addEdge(2, 3, -3);
57     bf.addEdge(2, 4, 9);
58     bf.addEdge(3, 1, -2);
59     bf.addEdge(4, 3, 7);
60     bf.addEdge(4, 0, 2);
61
62     int[] dist = bf.shortestPaths(0);
63     for (int i = 0; i < dist.length; i++) {
64         System.out.println("Distance from 0 to " + i + " = " + dist[i]);
65     }
66 }
67 }

```

Listing 7: Bellman–Ford Algorithm – Java

5.2 C Implementation

```

1  #include <stdio.h>
2  #include <limits.h>
3
4  #define MAX_E 100
5  #define INF INT_MAX
6
7  typedef struct {
8      int from, to, weight;
9  } Edge;
10
11 Edge edges[MAX_E];
12 int numEdges;
13 int numVertices;
14
15 void addEdge(int from, int to, int weight) {
16     edges[numEdges].from = from;
17     edges[numEdges].to = to;
18     edges[numEdges].weight = weight;
19     numEdges++;
20 }
21
22 void bellmanFord(int src) {
23     int dist[MAX_E];
24     for (int i = 0; i < numVertices; i++) dist[i] = INF;
25     dist[src] = 0;
26
27     // Relax all edges V - 1 times
28     for (int i = 1; i < numVertices; i++) {
29         for (int j = 0; j < numEdges; j++) {
30             int u = edges[j].from, v = edges[j].to, w = edges[j].weight;
31             if (dist[u] != INF && dist[u] + w < dist[v]) {
32                 dist[v] = dist[u] + w;

```

```
33     }
34 }
35 }
36
37 // Check for negative-weight cycles
38 for (int j = 0; j < numEdges; j++) {
39     int u = edges[j].from, v = edges[j].to, w = edges[j].weight;
40     if (dist[u] != INF && dist[u] + w < dist[v]) {
41         printf("Graph contains a negative-weight cycle\n");
42         return;
43     }
44 }
45
46 printf("Vertex \t Distance from Source %d\n", src);
47 for (int i = 0; i < numVertices; i++) {
48     if (dist[i] == INF)
49         printf("%d \t unreachable\n", i);
50     else
51         printf("%d \t %d\n", i, dist[i]);
52 }
53 }
54
55 int main(void) {
56     numVertices = 5;
57     numEdges = 0;
58
59     addEdge(0, 1, 6);
60     addEdge(0, 2, 7);
61     addEdge(1, 2, 8);
62     addEdge(1, 3, 5);
63     addEdge(1, 4, -4);
64     addEdge(2, 3, -3);
65     addEdge(2, 4, 9);
66     addEdge(3, 1, -2);
67     addEdge(4, 3, 7);
68     addEdge(4, 0, 2);
69
70     bellmanFord(0);
71     return 0;
72 }
```

Listing 8: Bellman–Ford Algorithm – C

6 Floyd–Warshall Algorithm

Floyd–Warshall computes shortest paths between **all pairs** of vertices using dynamic programming. For every intermediate vertex k , it checks whether routing through k shortens the path between every pair (i, j) .

Time Complexity	$O(V^3)$
Space Complexity	$O(V^2)$
Guarantees Shortest Path	Yes (no negative cycles)
Typical Use Case	Dense graphs, all-pairs shortest path, transitive closure

6.1 Java Implementation

```

1  import java.util.*;
2
3  public class FloydWarshall {
4
5      static final int INF = Integer.MAX_VALUE / 2; // avoid overflow on addition
6
7      public int[][] shortestPaths(int[][] graph) {
8          int n = graph.length;
9          int[][] dist = new int[n][n];
10         for (int i = 0; i < n; i++) {
11             dist[i] = Arrays.copyOf(graph[i], n);
12         }
13
14         // Try every vertex k as an intermediate point
15         for (int k = 0; k < n; k++) {
16             for (int i = 0; i < n; i++) {
17                 for (int j = 0; j < n; j++) {
18                     if (dist[i][k] + dist[k][j] < dist[i][j]) {
19                         dist[i][j] = dist[i][k] + dist[k][j];
20                     }
21                 }
22             }
23         }
24         return dist;
25     }
26
27     public static void main(String[] args) {
28         int n = 4;
29         int[][] graph = new int[n][n];
30         for (int[] row : graph) Arrays.fill(row, INF);
31         for (int i = 0; i < n; i++) graph[i][i] = 0;
32
33         graph[0][1] = 5;
34         graph[0][3] = 10;
35         graph[1][2] = 3;
36         graph[2][3] = 1;
37
38         FloydWarshall fw = new FloydWarshall();
39         int[][] dist = fw.shortestPaths(graph);
40
41         System.out.println("All-pairs shortest distances:");
42         for (int i = 0; i < n; i++) {
43             for (int j = 0; j < n; j++) {
44                 System.out.print((dist[i][j] >= INF ? "INF" : dist[i][j]) + "\t"
45 );

```

```
45         }
46         System.out.println();
47     }
48 }
49 }
```

Listing 9: Floyd–Warshall Algorithm – Java

6.2 C Implementation

```
1  #include <stdio.h>
2
3  #define V 4
4  #define INF 99999
5
6  void floydWarshall(int graph[V][V]) {
7      int dist[V][V];
8
9      for (int i = 0; i < V; i++)
10         for (int j = 0; j < V; j++)
11             dist[i][j] = graph[i][j];
12
13     // Try every vertex k as an intermediate point
14     for (int k = 0; k < V; k++) {
15         for (int i = 0; i < V; i++) {
16             for (int j = 0; j < V; j++) {
17                 if (dist[i][k] + dist[k][j] < dist[i][j]) {
18                     dist[i][j] = dist[i][k] + dist[k][j];
19                 }
20             }
21         }
22     }
23
24     printf("All-pairs shortest distances:\n");
25     for (int i = 0; i < V; i++) {
26         for (int j = 0; j < V; j++) {
27             if (dist[i][j] >= INF)
28                 printf("INF\t");
29             else
30                 printf("%d\t", dist[i][j]);
31         }
32         printf("\n");
33     }
34 }
35
36 int main(void) {
37     int graph[V][V] = {
38         {0,   5,   INF, 10},
39         {INF, 0,   3,   INF},
40         {INF, INF, 0,   1},
41         {INF, INF, INF, 0}
42     };
43
44     floydWarshall(graph);
45     return 0;
46 }
```

Listing 10: Floyd–Warshall Algorithm – C

7 A* Search Algorithm

A* is an informed search algorithm that extends Dijkstra's by combining the actual cost so far, $g(n)$, with a heuristic estimate of the remaining cost, $h(n)$, to reach the goal: $f(n) = g(n) + h(n)$. When $h(n)$ is **admissible** (never overestimates), A* is guaranteed to find the optimal path while typically exploring far fewer nodes than Dijkstra's.

Time Complexity	$O(E)$ in the best case, up to $O((V + E) \log V)$
Space Complexity	$O(V)$
Guarantees Shortest Path	Yes (with an admissible heuristic)
Typical Use Case	Grid/maze pathfinding, game AI, GPS navigation, robotics

7.1 Java Implementation

```

1  import java.util.*;
2
3  public class AStar {
4
5      static class Edge {
6          int to, weight;
7          Edge(int to, int weight) { this.to = to; this.weight = weight; }
8      }
9
10     static class Node {
11         int id, f;
12         Node(int id, int f) { this.id = id; this.f = f; }
13     }
14
15     private final Map<Integer, List<Edge>> graph = new HashMap<>();
16
17     public void addEdge(int u, int v, int weight) {
18         graph.computeIfAbsent(u, k -> new ArrayList<>()).add(new Edge(v, weight));
19         graph.computeIfAbsent(v, k -> new ArrayList<>()).add(new Edge(u, weight));
20         // undirected
21     }
22
23     // heuristic: estimated cost from node to goal (must be admissible)
24     public List<Integer> findPath(int start, int goal, int[] heuristic) {
25         Map<Integer, Integer> gScore = new HashMap<>();
26         Map<Integer, Integer> parent = new HashMap<>();
27         PriorityQueue<Node> openSet = new PriorityQueue<>(Comparator.comparingInt(n -> n.f));
28         Set<Integer> closed = new HashSet<>();
29
30         gScore.put(start, 0);
31         openSet.add(new Node(start, heuristic[start]));
32
33         while (!openSet.isEmpty()) {
34             Node current = openSet.poll();
35             if (current.id == goal) {
36                 return reconstructPath(parent, goal);
37             }
38             if (closed.contains(current.id)) continue;
39             closed.add(current.id);

```

```

40         for (Edge e : graph.getOrDefault(current.id, Collections.emptyList())
41             ) {
42             int tentativeG = gScore.get(current.id) + e.weight;
43             if (tentativeG < gScore.getOrDefault(e.to, Integer.MAX_VALUE)) {
44                 parent.put(e.to, current.id);
45                 gScore.put(e.to, tentativeG);
46                 int f = tentativeG + heuristic[e.to];
47                 openSet.add(new Node(e.to, f));
48             }
49         }
50         return Collections.emptyList(); // no path found
51     }
52
53     private List<Integer> reconstructPath(Map<Integer, Integer> parent, int goal
54     ) {
55         LinkedList<Integer> path = new LinkedList<>();
56         Integer at = goal;
57         while (at != null) {
58             path.addFirst(at);
59             at = parent.get(at);
60         }
61         return path;
62     }
63
64     public static void main(String[] args) {
65         AStar astar = new AStar();
66         astar.addEdge(0, 1, 1);
67         astar.addEdge(0, 2, 4);
68         astar.addEdge(1, 2, 2);
69         astar.addEdge(1, 3, 5);
70         astar.addEdge(2, 3, 1);
71
72         // Straight-line (admissible) heuristic estimate to goal node 3
73         int[] heuristic = {3, 2, 1, 0};
74
75         List<Integer> path = astar.findPath(0, 3, heuristic);
76         System.out.println("A* path: " + path);
77     }

```

Listing 11: A* Search – Java

7.2 C Implementation

```

1  #include <stdio.h>
2  #include <limits.h>
3
4  #define MAX_V 100
5  #define INF INT_MAX
6
7  int graph[MAX_V][MAX_V]; // adjacency matrix, 0 = no edge
8  int heuristic[MAX_V];    // admissible heuristic estimate to goal
9  int numVertices;
10
11 int findMinFOpen(int fScore[], int closed[]) {
12     int min = INF, minIndex = -1;
13     for (int v = 0; v < numVertices; v++) {
14         if (!closed[v] && fScore[v] < min) {
15             min = fScore[v];
16             minIndex = v;
17         }

```

```
18     }
19     return minIndex;
20 }
21
22 void printPath(int parent[], int node) {
23     if (parent[node] != -1) {
24         printPath(parent, parent[node]);
25         printf(" -> ");
26     }
27     printf("%d", node);
28 }
29
30 void aStar(int start, int goal) {
31     int gScore[MAX_V], fScore[MAX_V], parent[MAX_V], closed[MAX_V];
32
33     for (int i = 0; i < numVertices; i++) {
34         gScore[i] = INF;
35         fScore[i] = INF;
36         parent[i] = -1;
37         closed[i] = 0;
38     }
39
40     gScore[start] = 0;
41     fScore[start] = heuristic[start];
42
43     for (int count = 0; count < numVertices; count++) {
44         int current = findMinFOpen(fScore, closed);
45         if (current == -1) break; // no reachable nodes left
46
47         if (current == goal) {
48             printf("A* path: ");
49             printPath(parent, goal);
50             printf("\nTotal cost: %d\n", gScore[goal]);
51             return;
52         }
53
54         closed[current] = 1;
55
56         for (int neighbor = 0; neighbor < numVertices; neighbor++) {
57             if (graph[current][neighbor] && !closed[neighbor]) {
58                 int tentativeG = gScore[current] + graph[current][neighbor];
59                 if (tentativeG < gScore[neighbor]) {
60                     parent[neighbor] = current;
61                     gScore[neighbor] = tentativeG;
62                     fScore[neighbor] = tentativeG + heuristic[neighbor];
63                 }
64             }
65         }
66     }
67
68     printf("No path found from %d to %d\n", start, goal);
69 }
70
71 int main(void) {
72     numVertices = 4;
73     for (int i = 0; i < numVertices; i++)
74         for (int j = 0; j < numVertices; j++)
75             graph[i][j] = 0;
76
77     graph[0][1] = 1; graph[1][0] = 1;
78     graph[0][2] = 4; graph[2][0] = 4;
79     graph[1][2] = 2; graph[2][1] = 2;
80     graph[1][3] = 5; graph[3][1] = 5;
```

```
81     graph[2][3] = 1; graph[3][2] = 1;
82
83     // Admissible heuristic estimate to goal node 3
84     heuristic[0] = 3;
85     heuristic[1] = 2;
86     heuristic[2] = 1;
87     heuristic[3] = 0;
88
89     aStar(0, 3);
90     return 0;
91 }
```

Listing 12: A* Search – C

8 Summary & Algorithm Selection Guide

Algorithm	Best For	Time	Handles Neg. Weights	Optimal?
BFS	Unweighted graphs	$O(V + E)$	N/A	Yes
DFS	Reachability / any path	$O(V + E)$	N/A	No
Dijkstra	Weighted, non-negative	$O((V + E) \log V)$	No	Yes
Bellman–Ford	Weighted, may be negative	$O(VE)$	Yes	Yes
Floyd–Warshall	All pairs, dense graphs	$O(V^3)$	Yes	Yes
A*	Single pair, with heuristic	$\approx O(E)$	No	Yes*

Rule of thumb:

- Unweighted graph, need shortest path → **BFS**
- Just need any path / connectivity → **DFS**
- Weighted graph, all weights ≥ 0 → **Dijkstra's**
- Weighted graph, negative weights possible → **Bellman–Ford**
- Need shortest paths between every pair of nodes → **Floyd–Warshall**
- Know the goal location and have a good heuristic (e.g. grid/map) → **A***